

(DOM, Selecting Elements, Changing Content, Styles/Classes, Attributes, Create & Insert Elements, Remove Elements, Event Handling, Event Delegation, DOM Traversal, Form & Input Handling, Bubbling)

Here is a comprehensive overview of key DOM (Document Object Model) manipulation concepts and techniques in JavaScript, covering element selection, content and style changes, attribute handling, element creation/removal, event handling, event delegation, DOM traversal, form/input handling, and event bubbling:

Document Object Model (DOM)

The DOM is a programming interface representing the structure of HTML/XML documents as a tree of objects. It allows dynamic access and manipulation of the document's elements, attributes, and content^[1].

Selecting Elements

JavaScript provides multiple methods to select DOM elements:

- `getElementById(id)`: Selects a single element by its unique ID.
- `getElementsByClassName(className)`: Returns a live collection of elements matching the class.
- `getElementsByTagName(tagName)`: Returns a live collection of elements by tag name.
- `querySelector(selector)`: Returns the first element matching a CSS selector.
- `querySelectorAll(selector)`: Returns a static `NodeList` of all elements matching a CSS selector^[1].

Example:

```
const element = document.getElementById("myId");
const elements = document.querySelectorAll(".myClass");
```

Changing Content

You can modify the content of elements using:

- `element.innerHTML`: Get or set HTML content inside an element.
- `element.innerText` or `element.textContent`: Get or set plain text content.

Example:

```
document.getElementById("p1").innerHTML = "New text!";
```

Changing attribute values:

```
document.getElementById("myImage").src = "landscape.jpg";
```

or using

```
element.setAttribute("src", "landscape.jpg");
```

to set attributes dynamically^[2] ^[3].

Styles and Classes

Inline Styles

You can directly modify CSS properties via the `style` property of an element:

```
element.style.color = "red";  
element.style.fontSize = "20px";
```

Class Manipulation

The `classList` property provides methods to add, remove, toggle, or check classes:

```
element.classList.add("active");  
element.classList.remove("hidden");  
element.classList.toggle("open");
```

This approach is preferred over inline styles for maintainability^[4] ^[3].

Attributes vs Properties

- **Attributes** are defined in HTML and accessed/set via `getAttribute()` and `setAttribute()`.
- **Properties** are DOM object properties accessible directly (e.g., `element.id`, `element.value`).
- Attributes are always strings; properties can be various types.
- Not all attributes have corresponding properties and vice versa^[3].

Creating and Inserting Elements

You can create new elements using `document.createElement()` and insert them into the DOM with methods like:

- `parent.appendChild(newElement)`: Adds as last child.

- `parent.insertBefore(newElement, referenceElement)`: Inserts before a reference node.
- `element.insertAdjacentHTML(position, html)`: Inserts HTML at specified position (beforebegin, afterbegin, beforeend, afterend).

Example:

```
const newDiv = document.createElement("div");
newDiv.textContent = "Hello!";
document.body.appendChild(newDiv);
```

Removing Elements

To remove elements from the DOM:

- `element.remove()`: Removes the element from the DOM.
- `parent.removeChild(child)`: Removes a specified child node.

Example:

```
const element = document.getElementById("toRemove");
element.remove();
```

Event Handling

You can listen to events using:

- `element.addEventListener(event, handler)`: Attach event listeners.
- `element.removeEventListener(event, handler)`: Remove event listeners.

Example:

```
button.addEventListener("click", () => {
  alert("Button clicked!");
});
```

Event Delegation

Instead of attaching event listeners to multiple child elements, attach one listener to a common ancestor and use event bubbling to handle events from children:

```
parentElement.addEventListener("click", (event) => {
  if (event.target.matches(".child-class")) {
    // Handle click on child element
  }
});
```

This improves performance and simplifies code.

DOM Traversal

You can navigate the DOM tree using properties:

- `element.parentNode`: Get parent element.
- `element.children`: Get child elements.
- `element.nextElementSibling` / `element.previousElementSibling`: Get adjacent siblings.

Example:

```
const parent = element.parentNode;  
const firstChild = parent.children[0];
```

Form and Input Handling

Access and manipulate form inputs via properties like `.value`, `.checked` etc.

Example:

```
const input = document.querySelector("input[name='email']");  
console.log(input.value); // Get input value  
input.value = "new@example.com"; // Set input value
```

You can listen for form events like `submit` and input events like `change` or `input` to handle user interactions.

Event Bubbling

Events propagate from the target element up through ancestors (bubbling phase). This allows event delegation and centralized event handling.

You can stop propagation with:

```
event.stopPropagation();
```

This overview equips you with fundamental techniques to select, manipulate, style, create, remove, and handle events on DOM elements effectively in JavaScript, enabling dynamic and interactive web pages^{[1] [2] [4] [3] [5]}.



1. <https://codedamn.com/news/javascript/how-to-select-and-manipulate-dom-elements-in-javascript>
2. https://www.w3schools.com/js/js_htmlDOM_html.asp
3. <https://spinspire.com/learn/course/js-apps/dom/dom-props-attrs/>
4. <https://www.w3docs.com/learn-javascript/working-with-styles-in-dom.html>

5. <https://www.luisllamas.es/en/insert-and-remove-elements-from-the-dom-javascript/>